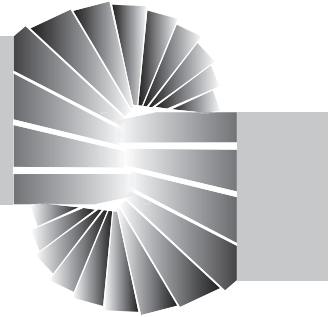




A unidade de controle

ASSUNTOS DA PARTE 4

Na Parte 3, concentramo-nos em instruções de máquina e operações efetuadas pelo processador para executar cada instrução. O que foi deixado de lado dessa discussão é exatamente como cada operação individual acontece. Esse é o trabalho da unidade de controle.

A unidade de controle é a parte do processador que faz com que as coisas aconteçam de fato. A unidade de controle emite sinais de controle externos ao processador para possibilitar a troca de dados com memória e módulos de E/S. A unidade de controle também emite sinais de controle internos ao processador para mover dados entre os registradores, para fazer com que ALU efetue uma determinada função e para regular outras operações internas. A entrada para unidade de controle consiste de registrador de instrução, flags e sinais de controle de fontes externas (por exemplo, sinais de interrupção).

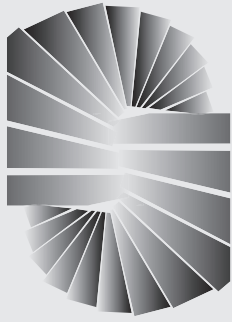
MAPA DA PARTE 4

Capítulo 15 Operação da unidade de controle

No Capítulo 15, nos voltamos para uma discussão sobre como as funções do processador são efetuadas ou, mais especificamente, como os vários elementos do processador são controlados para oferecer essas funções por meio da unidade de controle. É mostrado que cada ciclo de instrução é feito de um conjunto de micro-operações que gera sinais de controle. A execução é conseguida com o efeito desses sinais de controle que se originam na unidade de controle e vão para ALU, registradores e estrutura de interconexão do sistema. Finalmente, uma abordagem de uma implementação de unidade de controle, conhecida como implementação por hardware, é apresentada.

Capítulo 16 Controle microprogramado

No Capítulo 16, analisamos como o conceito de micro-operações leva a uma abordagem elegante e poderosa de uma implementação da unidade de controle conhecida como microprogramação. Basicamente, uma linguagem de programação de baixo nível é desenvolvida. Cada instrução na linguagem de máquina do processador é traduzida em uma sequência de instruções da unidade de controle de baixo nível. Essas instruções de baixo nível são conhecidas como microinstruções e o processo de tradução é conhecido como microprogramação. O capítulo descreve o layout de uma memória de controle contendo um microprograma para cada instrução de máquina que é descrita. A estrutura e a função da unidade de controle microprogramada então poderão ser explicadas.



Operação da unidade de controle

15.1 Micro-operações

- Ciclo de busca
- Ciclo indireto
- Ciclo de interrupção
- Ciclo de execução
- Ciclo de instrução

15.2 Controle do processador

- Requisitos funcionais
- Sinais de controle
- Exemplo de sinais de controle
- Organização interna do processador
- Intel 8085

15.3 Implementação por hardware

- Entradas da unidade de controle
- Lógica da unidade de controle

15.4 Leitura recomendada

PRINCIPAIS PONTOS

- A execução de uma instrução envolve a execução de uma sequência de subpassos, geralmente chamados de ciclos. Por exemplo, uma execução pode consistir de ciclos de busca, indireto, execução e interrupção. Cada ciclo é, por sua vez, feito de uma sequência de operações mais básicas chamadas de micro-operações. Uma única micro-operação geralmente envolve uma transferência entre registradores, uma transferência entre um registrador e um barramento externo ou uma simples operação da ALU.
 - A unidade de controle de um processador efetua duas tarefas: (1) ela faz com que o processador execute uma série de micro-operações na sequência correta, com base no programa que está sendo executado e (2) ela gera os sinais de controle que fazem com que cada micro-operação seja executada.
 - Os sinais de controle gerados pela unidade de controle causam a abertura e o fechamento de portas lógicas, resultando na transferência de dados para e de registradores e na operação da ALU.
- Uma técnica para implementar uma unidade de controle é conhecida como implementação por hardware (*hardwired*), onde a unidade de controle é um circuito combinatório. Seus sinais lógicos de entrada, governados pela instrução de máquina corrente, são transferidos para um conjunto de sinais de controle de saída.

No Capítulo 10, destacamos que um conjunto de instruções de máquina nos leva à definição de um processador. Se conhecemos o conjunto de instruções de máquina, incluindo o entendimento do efeito de cada *opcode* e o entendimento de modos de endereçamento, e se conhecemos o conjunto de registradores visíveis ao usuário, então sabemos as funções que o processador deve efetuar. Esta não é uma imagem completa. Precisamos conhecer as interfaces externas, normalmente por meio de um barramento, e como as interrupções são tratadas. Com esta linha de raciocínio, surge a seguinte lista de itens necessários para especificar o funcionamento de um processador:

1. Operações (*opcode*).
2. Modos de endereçamento.
3. Registradores.
4. Interface com módulos de E/S.

5. Interface com o módulo de memória.
6. Interrupções.

Esta lista, embora generalizada, é bastante completa. Os itens de 1 até 3 são definidos pelo conjunto de instruções. Os itens 4 e 5 são normalmente definidos ao se especificar o barramento do sistema. E o item 6 é definido parcialmente pelo barramento do sistema e parcialmente pelo tipo de suporte que o processador oferece ao sistema operacional.

Esta lista de seis itens pode ser denominada como os requisitos funcionais para um processador. Eles determinam o que um processador tem que fazer. Foi isso o que nos ocupou nas partes dois e três. Agora nos voltamos para a questão de como essas funções são efetuadas ou, mais especificamente, como os vários elementos do processador são controlados para oferecer essas funções. Assim chegamos à discussão sobre a unidade de controle, a qual controla a operação do processador.



15.1 Micro-operações

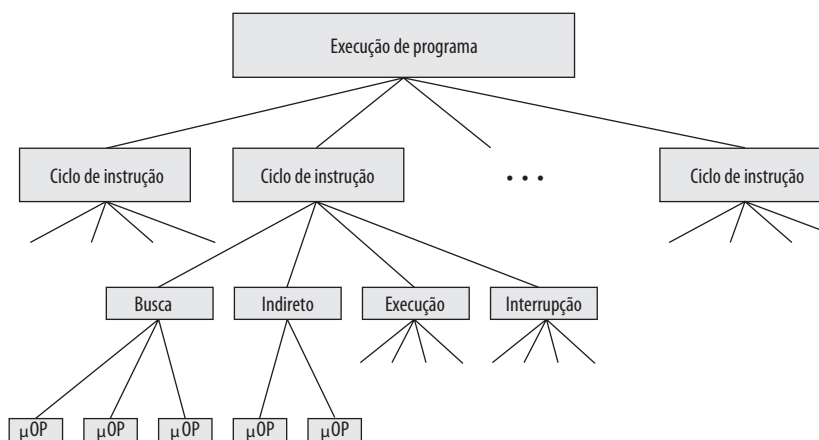
Vimos que a operação de um computador, ao executar um programa, consiste de uma sequência de ciclos de instrução, com uma instrução de máquina por ciclo. É claro que devemos lembrar que essa sequência de ciclos de instruções não é necessariamente a mesma que a *sequência da escrita* das instruções no programa por causa da existência de instruções de desvio. Aqui nos referimos à sequência de instruções em tempo de execução.

Vimos também que cada ciclo de instrução é feito de um número de pequenas unidades. Uma subdivisão conveniente é busca, indireto, execução e interrupção, sendo que apenas os ciclos de busca e execução sempre ocorrem.

No entanto, para projetar uma unidade de controle, precisamos detalhar ainda mais a descrição. Na nossa discussão sobre pipeline no Capítulo 12, nós começamos a ver que uma decomposição maior é possível. Na verdade, veremos que cada um dos ciclos menores envolve uma série de passos, onde cada um destes envolve os registradores do processador. Iremos nos referir a esses passos como *micro-operações*. O prefixo *micro* refere-se ao fato de que cada passo é muito simples e realiza muito pouco. A Figura 15.1 ilustra a relação entre vários conceitos que estivemos discutindo. Para resumir, a execução de um programa consiste da execução sequencial de instruções. Cada instrução é executada durante um ciclo de instrução feito de subciclos menores (por exemplo, busca, indireto, execução, interrupção). A execução de cada subciclo envolve uma ou mais operações mais curtas, ou seja, micro-operações.

As micro-operações são operações funcionais, ou atômicas, de um processador. Nesta seção, analisamos micro-operações para obter um entendimento sobre como os eventos de um ciclo de instrução podem ser descritos como uma sequência dessas micro-operações. Um exemplo simples será usado. No restante deste

Figura 15.1 Elementos que constituem uma execução de programa



capítulo mostramos como o conceito de micro-operações serve como uma orientação para projetar uma unidade de controle.



Ciclo de busca

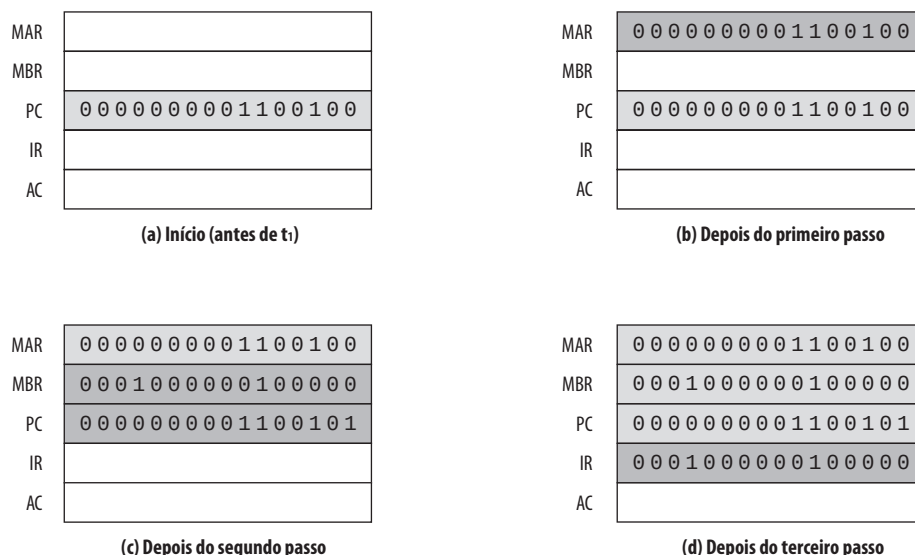
Começamos com a análise do ciclo de busca, o qual ocorre no início de cada ciclo de instrução e faz com que uma instrução seja obtida da memória. Para propósitos desta discussão assumimos a organização ilustrada na Figura 12.6. Quatro registradores estão envolvidos:

- **Registrador de endereço de memória (MAR):** conectado às linhas de endereço do barramento de sistema. Ele especifica o endereço na memória para uma operação de leitura ou escrita.
- **Registrador de buffer de memória (MBR):** conectado às linhas de dados do barramento de sistema. Ele contém o valor a ser guardado na memória ou o último valor lido da memória.
- **Contador de programa (PC):** guarda o endereço da próxima instrução a ser lida.
- **Registrador de instrução (IR):** guarda a última instrução lida.

Vamos olhar a sequência de eventos para ciclo de leitura do ponto de vista dos seus efeitos nos registradores do processador. Um exemplo aparece na Figura 15.2. No início do ciclo de instrução, o endereço da próxima instrução a ser executada está no contador de programa (PC); neste caso, o endereço é 1100100. O primeiro passo é mover esse endereço para o registrador de endereço de memória (MAR) porque este é o único registrador conectado às linhas de endereços do barramento de sistema. O segundo passo é trazer a instrução. O endereço desejado (em MAR) é colocado no barramento de endereços, a unidade de controle emite um comando READ no barramento de controle e o resultado aparece no barramento de dados e é copiado para o registrador de buffer de memória (MBR). Precisamos também incrementar PC pelo tamanho da instrução para se preparar para a próxima instrução. Como estas duas ações (ler da memória e incrementar PC) não interferem uma com a outra, podemos executá-las simultaneamente para economizar tempo. O terceiro passo é mover o conteúdo de MBR para registrador de instrução (IR). Isto libera MBR para uso durante um possível ciclo indireto.

Assim, o simples ciclo de leitura na prática consiste de três passos e quatro micro-operações. Cada micro-operação envolve movimentação de dados para dentro ou fora de um registrador. Enquanto essas movimentações não interferem umas nas outras, várias delas podem ocorrer durante um passo, economizando tempo. Simbolicamente, podemos escrever esta sequência de eventos da seguinte forma:

Figura 15.2 Sequência de eventos, ciclo de leituras



$$\begin{aligned} t_1: \text{MAR} &\leftarrow (\text{PC}) \\ t_2: \text{MBR} &\leftarrow \text{Memory} \\ &\text{PC} \leftarrow (\text{PC}) + I \\ t_3: \text{IR} &\leftarrow (\text{MBR}) \end{aligned}$$

onde I é o tamanho da instrução. Precisamos fazer vários comentários sobre esta sequência. Assumimos que um clock está disponível para propósitos de tempo e que ele emite pulsos de clock em intervalos regulares. Cada pulso de clock define uma unidade de tempo. Assim, todas as unidades de tempo são de duração igual. Cada micro-operação pode ser efetuada dentro do tempo de uma única unidade de tempo. A notação (t_1, t_2, t_3) representa unidades de tempo sucessivas. Desta forma temos:

- **Primeira unidade de tempo:** move conteúdo de PC para MAR.
- **Segunda unidade de tempo:** move conteúdo da posição de memória especificada por MAR para MBR. Incrementado em I o conteúdo de PC.
- **Terceira unidade de tempo:** move o conteúdo de MBR para IR.

Observe que a segunda e a terceira micro-operações ocorrem durante a segunda unidade de tempo. A terceira micro-operação poderia ser agrupada com a quarta sem afetar a operação de leitura:

$$\begin{aligned} t_1: \text{MAR} &\leftarrow (\text{PC}) \\ t_2: \text{MBR} &\leftarrow \text{Memory} \\ t_3: \text{PC} &\leftarrow (\text{PC}) + I \\ &\text{IR} \leftarrow (\text{MBR}) \end{aligned}$$

O agrupamento de micro-operações deve seguir duas regras simples:

1. A sequência de eventos apropriada deve ser seguida. Assim $(\text{MAR} \leftarrow (\text{PC}))$ deve preceder $(\text{MBR} \leftarrow \text{Memória})$ porque as operações de leitura de memória usam o endereço em MAR.
2. Conflitos devem ser evitados. Não se deve tentar ler e escrever no mesmo registrador em uma unidade de tempo, porque os resultados seriam imprevisíveis. Por exemplo, as micro-operações $(\text{MBR} \leftarrow \text{Memória})$ e $(\text{IR} \leftarrow \text{MBR})$ não devem ocorrer durante a mesma unidade de tempo.

Um último ponto que deve ser observado é relacionado com as micro-operações que envolvem uma adição. Para evitar a duplicidade de circuitos, essa adição poderia ser efetuada pela ALU. O uso da ALU pode envolver micro-operações adicionais, dependendo da funcionalidade da ALU e da organização do processador. Deixamos a discussão sobre este ponto para mais à frente neste capítulo.

É útil comparar eventos descritos nesta e nas próximas subseções com a Figura 3.5. Apesar das micro-operações serem ignoradas nessa figura, esta discussão mostra as micro-operações necessárias para efetuar os subciclos do ciclo de instrução.



Ciclo indireto

Uma vez lida a instrução, o próximo passo é buscar os operandos fontes. Continuando o nosso exemplo simples, imaginemos um formato da instrução com um endereço, com endereçamento direto e indireto permitido. Se a instrução especifica um endereço indireto, então um ciclo indireto deve preceder o ciclo de execução. O fluxo de dados difere um pouco do mostrado na Figura 12.7 e inclui as seguintes micro-operações:

$$\begin{aligned} t_1: \text{MAR} &\leftarrow (\text{IR}(\text{Endereço})) \\ t_2: \text{MBR} &\leftarrow \text{Memória} \\ t_3: \text{IR}(\text{Endereço}) &\leftarrow (\text{MBR}(\text{Endereço})) \end{aligned}$$

O campo de endereço da instrução é transferido para MAR. Este é então usado para obter o endereço do operando. Finalmente, o campo de endereço de IR é atualizado a partir de MBR, então agora ele contém um endereço direto em vez de um indireto.

O IR agora está no mesmo estado como se endereçamento indireto não tivesse sido usado e está pronto para o ciclo de execução. Pulamos esse ciclo por um momento para analisar o ciclo de interrupção.



Ciclo de interrupção

Ao completar o ciclo de execução, um teste é feito para determinar se houve qualquer interrupção habilitada. Se sim, ocorre o ciclo de interrupção. A natureza deste ciclo varia muito de uma máquina para outra. Apresentamos uma sequência muito simples de eventos, conforme ilustrado na Figura 12.8. Temos

```
t1: MBR ← (PC)
t2: MAR ← Endereço_salvar
    PC ← Endereço_rotina
t3: Memória ← (MBR)
```

No primeiro passo, o conteúdo de PC é transferido para MBR, para que ele possa ser salvo para o retorno da interrupção. O MAR é então carregado com o endereço onde o conteúdo de PC deve ser salvo e o PC é carregado com o endereço do início da rotina do tratamento de interrupção. Cada uma destas duas ações pode ser uma única micro-operação. No entanto, como a maioria dos processadores fornece vários tipos e/ou níveis de interrupções, pode levar uma ou mais micro-operações adicionais para obter Endereço_Salvar e Endereço_Rotina antes que eles possam ser transferidos para MAR e PC, respectivamente. De qualquer forma, uma vez feito isso, o passo final é armazenar MBR, o qual contém o valor antigo de PC, em memória. O processador está pronto agora para começar o próximo ciclo de instrução.



Ciclo de execução

Os ciclos de busca, indireto e interrupção são simples e previsíveis. Cada um deles envolve uma sequência pequena e fixa de micro-operações e, em cada caso, as mesmas micro-operações são repetidas a cada vez.

Isso não é verdade para o ciclo de execução. Por causa da variedade de *opcodes*, existe uma série de diferentes sequências de micro-operações que podem ocorrer. Vamos considerar alguns exemplos hipotéticos.

Primeiro, considere uma instrução de soma:

ADD R1, X

a qual adiciona o conteúdo da posição X para o registrador R1. A seguinte sequência de micro-operações pode ocorrer:

```
t1: MAR ← (IR(endereço))
t2: MBR ← Memória
t3: R1 ← (R1) + (MBR)
```

Começamos com IR contendo a instrução ADD. No primeiro passo, a parte de endereço de IR é carregado em MAR. Então a posição de memória referenciada é lida. Finalmente, os conteúdos de R1 e MBR são adicionados por ALU. Novamente, este é um exemplo simplificado. Micro-operações adicionais podem ser necessárias para extrair a referência do registrador de IR e talvez para armazenar entradas e saídas da ALU em alguns registradores intermediários.

Vamos olhar dois exemplos mais complexos. Uma instrução comum é incrementar e pular se zero:

ISZ X

O conteúdo da posição X é incrementado por 1. Se o resultado for 0, a próxima instrução é pulada. Uma sequência possível de micro-operações é

```
t1: MAR ← (IR(endereço))
t2: MBR ← Memória
t3: MBR ← (MBR) + 1
t4: Memória ← (MBR)
    se ((MBR) = 0) então (PC ← (PC) + 1)
```

Um novo recurso introduzido aqui é a ação condicional. O PC é incrementado se (MBR) = 0. Este teste e ação podem ser implementados com uma micro-operação. Observe também que esta micro-operação pode ser efetuada durante a mesma unidade de tempo em que o valor atualizado em MBR está sendo armazenado de volta em memória.

Finalmente, considere uma instrução de chamada de sub-rotina chamando instrução. Considere como exemplo uma instrução de desvio-e-salvar-endereço:

BSA X

O endereço da instrução que segue a instrução BSA é salvo na posição X e a execução continua na posição X + I. O endereço salvo será usado posteriormente para retorno. Esta é uma técnica simples para permitir chamadas de sub-rotinas. A seguinte micro-operação é suficiente:

```
t1: MAR ← (IR(endereço))  
    MBR ← (PC)  
t2: PC ← (IR(endereço))  
    Memória ← (MBR)  
t3: PC ← (PC) + I
```

O endereço em PC no início da instrução é o endereço da próxima instrução na sequência. Isto é salvo no endereço designado em IR. O endereço posterior é incrementado também para fornecer o endereço da instrução para o próximo ciclo de instrução.



Ciclo de instrução

Vimos que cada fase do ciclo de instrução pode ser decomposto em uma sequência de micro-operações básicas. No nosso exemplo há uma sequência para cada ciclo de busca, indireto e interrupção e para o ciclo de execução há uma sequência de micro-operações para cada *opcode*.

Para completar o quadro precisamos ligar as sequências de micro-operações juntas e isso é feito na Figura 15.3. Supomos um novo registrador de 2 bits chamado de código de ciclo de instrução (ICC). O ICC define o estado do processador, ou seja, em qual parte do ciclo o mesmo se encontra:

- 00: Busca.
- 01: Indireto.
- 10: Execução.
- 11: Interrupção.

No fim de cada um dos quatro ciclos, ICC é definido apropriadamente. O ciclo indireto é sempre seguido de ciclo de execução. O ciclo de interrupção é sempre seguido do ciclo de busca (veja Figura 12.4). Para ciclo de leitura e para o de execução, o próximo ciclo depende do estado do sistema.

Assim, o fluxograma da Figura 15.3 define a sequência completa de micro-operações, dependendo apenas da sequência de instruções e do padrão de interrupção. É claro que este é um exemplo simplificado. O fluxograma para um processador real seria mais complexo. De qualquer maneira, nós alcançamos o ponto em nossa discussão onde a operação do processador é definida como desempenho de uma sequência de micro-operações. Podemos agora considerar como a unidade de controle faz com que essa sequência ocorra.



15.2 Controle do processador

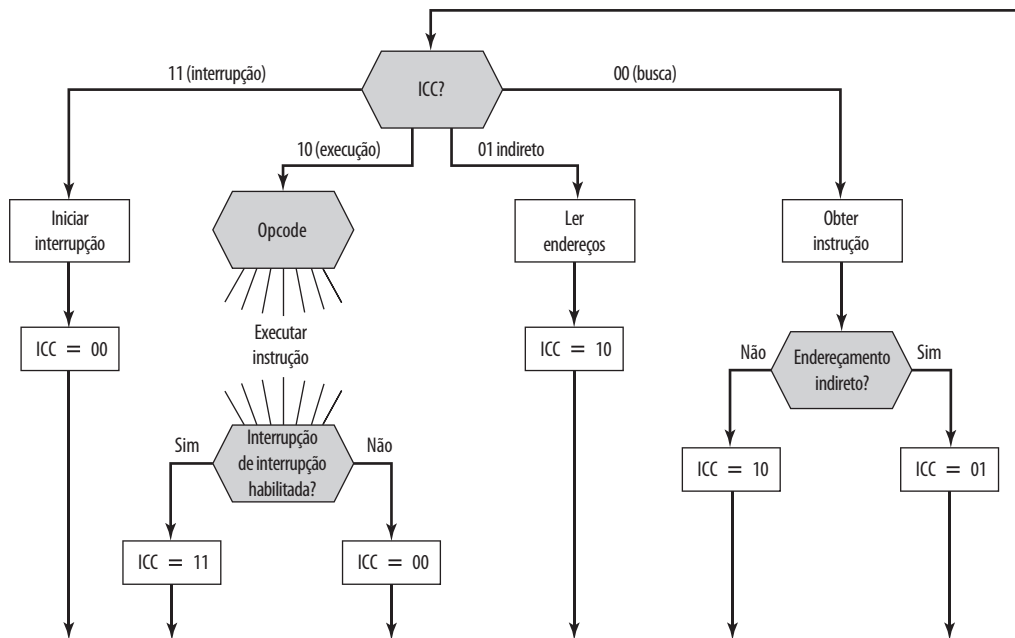


Requisitos funcionais

Como resultados das nossas análises da seção anterior, decomposemos o comportamento ou o funcionamento do processador em operações básicas, chamadas de micro-operações. Ao reduzir a operação do processador para o seu nível mais básico, somos capazes de definir o que exatamente a unidade de controle deve fazer acontecer. Assim, podemos definir os *requisitos funcionais* para a unidade de controle: aquelas funções que a unidade de controle tem que executar. Uma definição desses requisitos funcionais é a base para o projeto e a implementação da unidade de controle.

Com a informação em mãos, o próximo processo de três passos leva a uma caracterização da unidade de controle:

1. Definir elementos básicos do processador.
2. Descrever as micro-operações que o processador executa.
3. Determinar as funções que a unidade de controle deve realizar para fazer com que as micro-operações sejam executadas.

Figura 15.3 Fluxograma do ciclo de instrução

Nós já fizemos os passos 1 e 2. Vamos resumir o resultado. Primeiro, os elementos funcionais básicos do processador são os seguintes:

- ALU.
- Registradores.
- Caminhos de dados internos.
- Caminhos de dados externos.
- Unidade de controle.

Alguma reflexão deverá convencê-lo de que esta lista está completa. A ALU é a essência funcional do computador. Os registradores são usados para armazenar os dados internamente ao processador. Alguns registradores contêm a informação de estado necessária para gerenciar o sequenciamento de instruções (por exemplo, uma palavra de estado de programa). Outros contêm os dados que vêm de ou vão para ALU, memória e módulos de E/S. Caminhos de dados internos são usados para mover dados entre registradores e entre registrador e ALU. Caminhos de dados externos ligam os registradores com memória e módulos de E/S, frequentemente por meio de um barramento do sistema. A unidade de controle faz com que as operações aconteçam dentro do processador.

A execução de um programa consiste em operações envolvendo esses elementos do processador. Conforme vimos, essas operações consistem em uma sequência de micro-operações. Depois de rever a Seção 15.1, o leitor deveria perceber que todas as micro-operações se enquadram em uma das categorias a seguir:

- Transferência de dados de um registrador para outro.
- Transferência de dados de um registrador para uma interface externa (por exemplo, barramento do sistema).
- Transferência de dados de uma interface externa para um registrador.
- Efetuar uma operação aritmética ou lógica, usando registradores para entrada e saída.

Todas as micro-operações necessárias para efetuar um ciclo de instrução, incluindo todas as micro-operações para executar cada instrução dentro do conjunto de instruções, encaixam-se em uma dessas categorias.

Podemos agora ser mais explícitos sobre a maneira como funciona a unidade de controle. A unidade de controle desempenha duas tarefas básicas:

- **Sequenciamento:** a unidade de controle faz o processador executar por uma série de micro-operações na sequência correta, com base no programa que está sendo executado.
- **Execução:** a unidade de controle faz cada micro-operação ser executada.

O que precede é uma descrição funcional sobre o que a unidade de controle faz. A chave para o funcionamento da unidade de controle é o uso de sinais de controle.



Sinais de controle

Nós definimos os elementos que fazem o processador (ALU, registradores, caminhos de dados) e as micro-operações que são executadas. Para que a unidade de controle desempenhe a sua função, ela precisa ter entradas que lhe permitam determinar o estado do sistema e saídas que lhe permitam controlar o comportamento do sistema. Essas são as especificações externas da unidade de controle. Internamente, a unidade de controle precisa ter a lógica necessária para desempenhar as suas funções de sequenciamento e execução. Adiamos a discussão sobre a operação interna da unidade de controle para a Seção 15.3 e o Capítulo 16. O restante desta seção concentra-se na interação entre a unidade de controle e outros elementos do processador.

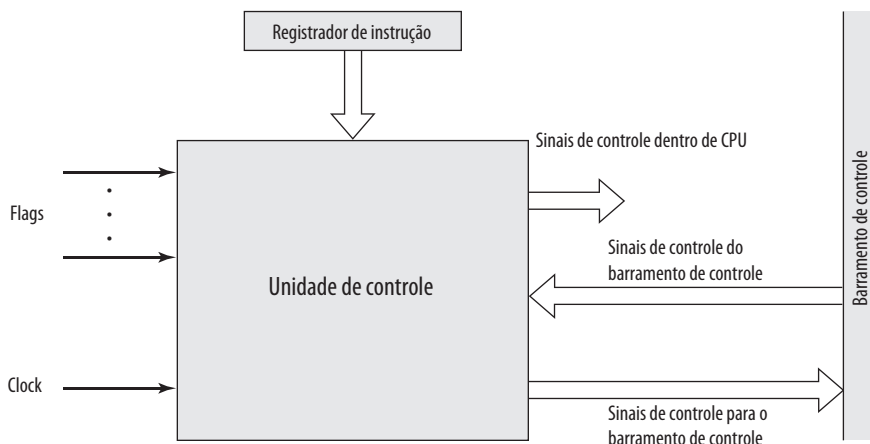
A Figura 15.4 é um modelo geral da unidade de controle, mostrando todas as suas entradas e saídas. As entradas são:

- **Clock:** é como a unidade de controle “mantém o tempo”. A unidade de controle faz uma micro-operação (ou um conjunto de micro-operações simultâneas) ser executada em cada pulso de clock. Isso, às vezes é chamado de tempo de ciclo do processador ou tempo de ciclo do clock.
- **Registrador de instrução:** o *opcode* e o modo de endereçamento da instrução corrente são usados para determinar qual micro-operação executar durante o ciclo de execução.
- **Flags:** estas são necessárias para a unidade de controle determinar o estado do processador e das saídas das operações anteriores da ALU. Por exemplo, para a instrução incrementar-e-pular-se-zero (ISZ), a unidade de controle irá incrementar PC se o flag zero tiver valor igual a 1.
- **Sinais de controle do barramento de controle:** barramento de controle é uma parte do barramento de sistema que fornece sinais para a unidade de controle.

As saídas são:

- **Sinais de controle dentro do processador:** existem dois tipos: aqueles que fazem os dados serem movidos de um registrador para outro e aqueles que ativam as funções específicas da ALU.
- **Sinais de controle para barramento de controle:** existem dois tipos também: sinais de controle para memória e sinais de controle para módulos de E/S.

Figura 15.4 Diagrama de blocos da unidade de controle



Três tipos de sinais de controle são usados: os que ativam uma função da ALU, os que ativam um caminho de dados e os que são sinais para o barramento externo do sistema ou para outra interface externa. Todos estes sinais no final das contas são aplicados como entradas binárias para portas lógicas individuais.

Vamos considerar novamente o ciclo de busca para ver como a unidade de controle mantém o controle. A unidade de controle mantém a informação sobre onde está dentro do ciclo de instrução. Em um determinado ponto, ela sabe que o ciclo de leitura será executado a seguir. O primeiro passo é transferir o conteúdo de PC para MAR. A unidade de controle faz isso ativando o sinal de controle que abre as portas lógicas entre os bits de PC e bits de MAR. O próximo passo é ler uma palavra da memória para dentro de MBR e incrementar PC. A unidade de controle faz isso enviando os seguintes sinais de controle simultaneamente:

- Um sinal de controle que abre portas lógicas, permitindo que o conteúdo de MAR seja transferido para o barramento de endereços.
- Um sinal de controle de leitura de memória é colocado no barramento de controle.
- Um sinal de controle que abre as portas, permitindo que o conteúdo do caminho de dados seja armazenado em MBR.
- Sinais de controle para lógica que adicionam 1 ao conteúdo de PC e armazenam o resultado de volta em PC.

Depois disso, a unidade de controle envia um sinal de controle que abre portas lógicas entre MBR e IR.

Isso completa o ciclo de leitura exceto para uma coisa: a unidade de controle precisa decidir se executa um ciclo indireto ou um ciclo de execução a seguir. Para decidir isso, ela examina IR para ver se uma referência indireta de memória é feita.

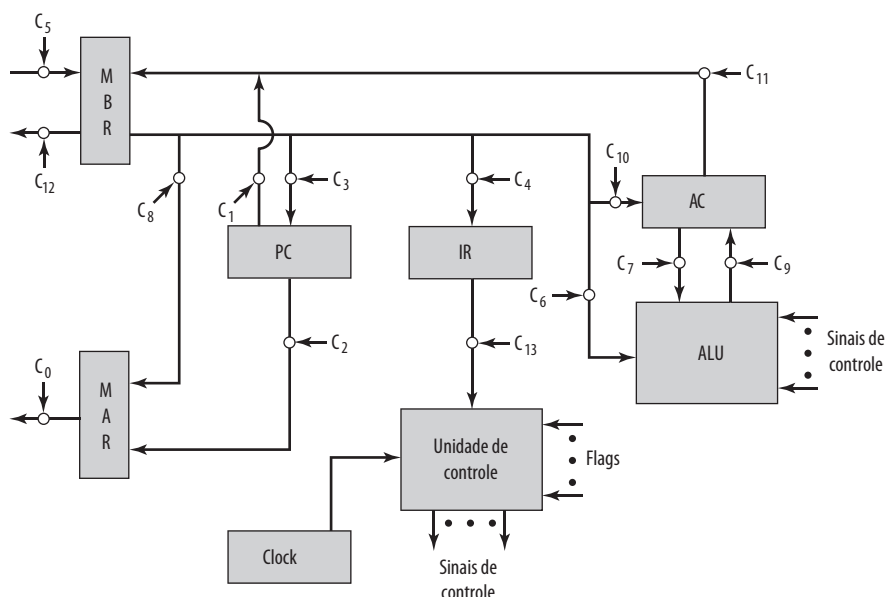
O ciclo indireto e o de interrupção funcionam de maneira semelhante. Para o ciclo de execução, a unidade de controle começa examinando o *opcode* e, com base nisso, decide qual sequência de micro-operações deve executar no ciclo de execução.



Exemplo de sinais de controle

Para ilustrar o funcionamento da unidade de controle, vamos analisar um exemplo simples. A Figura 15.5 ilustra o exemplo. Trata-se de um processador simples com um único acumulador (AC). Os caminhos de dados entre os elementos estão indicados. Os caminhos de controle para sinais que se originam da unidade de controle não são mostrados, porém as terminações dos sinais de controle são marcadas como C_i e indicadas por um círculo.

Figura 15.5 Caminhos de dados e sinais de controle



A unidade de controle recebe entradas a partir do clock, do registrador de instrução e flags. A cada ciclo de clock, a unidade de controle lê todas as suas entradas e emite um conjunto de sinais de controle. Os sinais de controle vão para três destinos diferentes:

- **Caminhos de dados:** a unidade de controle controla o fluxo interno de dados. Por exemplo, durante a leitura da instrução, o conteúdo do registrador de buffer de memória (MBR) é transferido para o registrador de instrução. Para cada caminho a ser controlado, há uma chave (indicado por um círculo na figura). Um sinal de controle da unidade de controle abre as portas temporariamente para deixar os dados passarem.
- **ALU:** a unidade de controle controla a operação de ALU por meio de um conjunto de sinais de controle. Esses sinais ativam vários circuitos e portas lógicas dentro da ALU.
- **Barramento de sistema:** a unidade de controle envia sinais de controle para linhas de controle do barramento de sistema (por exemplo, READ para leitura de memória).

A unidade de controle deve manter a informação sobre onde está dentro do ciclo de instrução. Usando esse conhecimento e lendo todas as suas entradas, a unidade de controle emite uma sequência de sinais de controle que fazem com que as micro-operações ocorram. Ela usa pulsos de clock para temporizar a sequência de eventos, permitindo um tempo entre eventos para os níveis de sinal estabilizarem-se. A Tabela 15.1 indica os sinais de controle que são necessários para algumas sequências de micro-operações descritas anteriormente. Por simplicidade, os caminhos de dados e de controle para incrementar PC e para carregar endereços fixos em PC e MAR não são mostrados.

Vale a pena analisar a natureza mínima da unidade de controle. Ela é a máquina que faz funcionar todo o computador inteiro. E faz isso sabendo apenas as instruções a serem executadas e a natureza dos resultados das operações aritméticas e lógicas (por exemplo, positivo, *over-flow* etc.). Ela nunca verifica os dados sendo processados ou os resultados produzidos, e ela controla tudo com poucos sinais de controle dentro do processador e no barramento de sistema.



Organização interna do processador

A Figura 15.5 indica o uso de vários caminhos de dados. A complexidade desse tipo de organização deve ficar clara. Normalmente algum tipo de arranjo de barramento interno, como o sugerido na Figura 12.2, será usado.

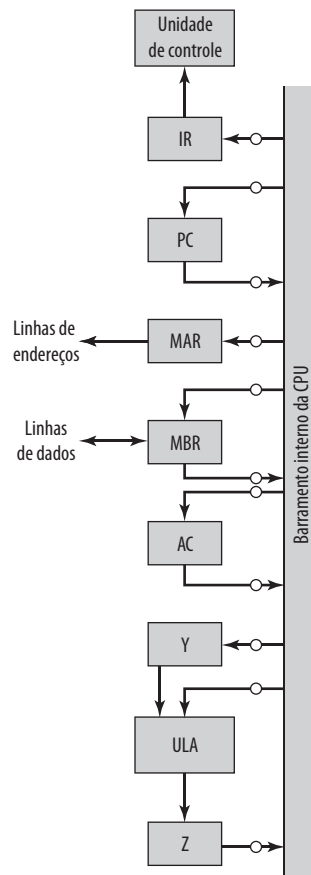
Usando um barramento interno do processador, a Figura 15.5 pode ser rearranjada conforme mostrado na Figura 15.6. Um único barramento interno conecta a ALU e todos os registradores do processador.

Tabela 15.1 Micro-operações e sinais de controle

	Micro-operações	Sinais de controle ativos
Busca:	$t_1: \text{MAR} \leftarrow (\text{PC})$	C_2
	$t_2: \text{MBR} \leftarrow \text{Memória}$ $\text{PC} \leftarrow (\text{PC}) + 1$	C_5, C_R
	$t_3: \text{IR} \leftarrow \text{MBR}$	C_4
Indireto:	$t_1: \text{MAR} \leftarrow (\text{IR}(\text{Endereço}))$	C_8
	$t_2: \text{MBR} \leftarrow \text{Memória}$	C_5, C_R
	$t_3: \text{IR}(\text{Endereço}) \leftarrow (\text{MBR}(\text{Endereço}))$	C_4
Interrupção	$t_1: \text{MBR} \leftarrow (\text{PC})$	C_1
	$t_2: \text{MAR} \leftarrow \text{Endereço-salvar}$ $\text{PC} \leftarrow \text{Endereço-rotina}$	
	$t_3: \text{Memória}(\text{MBR})$	C_{12}, C_W

C_R = Sinal de controle de leitura para o barramento de sistema.

C_W = Sinal de controle de escrita para o barramento de sistema.

Figura 15.6 CPU com barramento interno

Portas e sinais de controle são disponibilizados para a movimentação de dados do barramento para cada registrador e vice-versa. Sinais de controle adicionais controlam a transferência de dados para e do barramento do sistema (externo) e a operação da ALU.

Dois novos registradores, definidos como Y e Z, foram adicionados à organização. Estes são necessários para a correta operação da ALU. Quando uma operação que envolve dois operandos é executada, um pode ser obtido do barramento interno, mas outro tem que ser obtido de outra fonte. O AC pode ser usado para esse propósito, mas isso limita a flexibilidade do sistema e não funcionaria com um processador com vários registradores de propósito geral. O registrador Y permite armazenamento temporário para outra entrada. A ALU é um circuito combinatório (veja o Capítulo 20) sem nenhum local de armazenamento interno. Assim, quando o sinal de controle ativa uma função da ALU, a entrada para a ALU é transformada em saída. Assim, a saída da ALU não pode ser conectada diretamente ao barramento, porque essa saída alimentaria de volta a entrada. O registrador Z permite o armazenamento temporário de saída. Com esse arranjo, uma operação para adicionar um valor da memória para AC teria os seguintes passos:

```

t1: MAR ← (IR(endereço))
t2: MBR ← Memória
t3: Y ← (MBR)
t4: Z ← (AC) + (Y)
t5: AC ← (Z)

```

Outras organizações são possíveis, mas em geral algum tipo de barramento interno ou conjunto de barramentos internos é usado. O uso de caminhos de dados comuns simplifica o layout de interconexão e o controle do processador. Outra razão prática para uso de um barramento interno é para economizar espaço.



Intel 8085

Para ilustrar alguns dos conceitos introduzidos até agora neste capítulo, vamos analisar o Intel 8085. A sua organização é mostrada na Figura 15.7. Vários componentes-chave que podem não ser autoexplicativos são:

- **Latch incrementador/decrementador de endereço:** lógica que pode adicionar 1 a ou subtrair 1 de conteúdo do ponteiro de pilha ou contador de programa. Isto economiza tempo evitando o uso de ALU para esse propósito.
- **Controle de interrupção:** este módulo lida com múltiplos níveis de sinais de interrupção.
- **Controle E/S serial:** este módulo é uma interface para os dispositivos que comunicam 1 bit por vez.

A Tabela 15.2 descreve os sinais externos para dentro e fora do 8085. Eles são ligados com o barramento do sistema externo. Esses sinais são a interface entre o processador 8085 e o restante do sistema (Figura 15.8).

A unidade de controle é identificada como tendo dois componentes chamados de (1) decodificador de instrução e codificação de ciclo de máquina e (2) temporização e controle. A discussão sobre o primeiro é adiada para a próxima seção. A essência da unidade de controle é o módulo de temporização e controle. Este módulo inclui um relógio e aceita como entradas a instrução atual e alguns sinais de controle externos. As suas saídas consistem de sinais de controle para outros componentes do processador mais sinais de controle para o barramento de sistema externo.

Figura 15.7 Diagrama de blocos da CPU Intel 8085

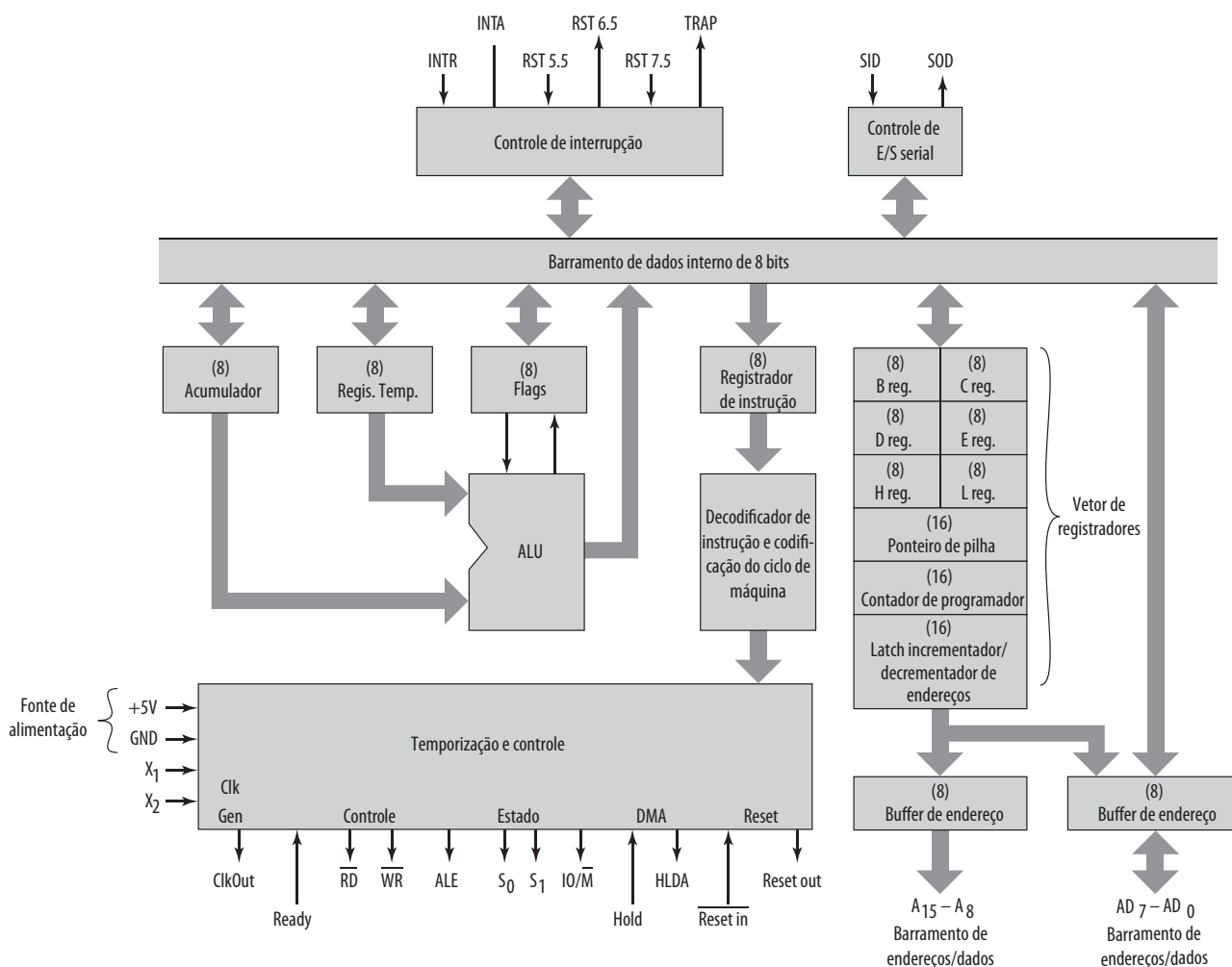
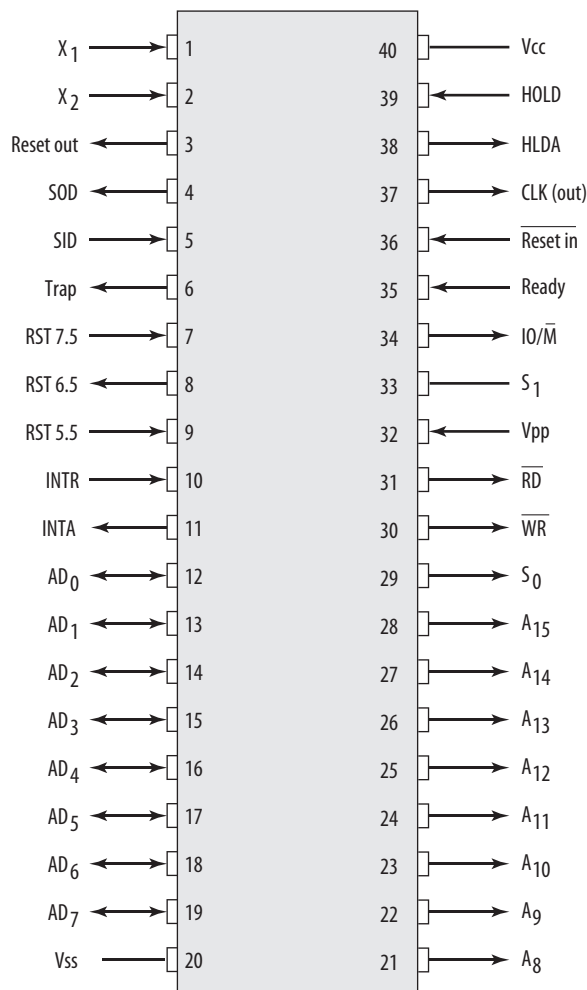


Tabela 15.2 Sinais externos de Intel 8085

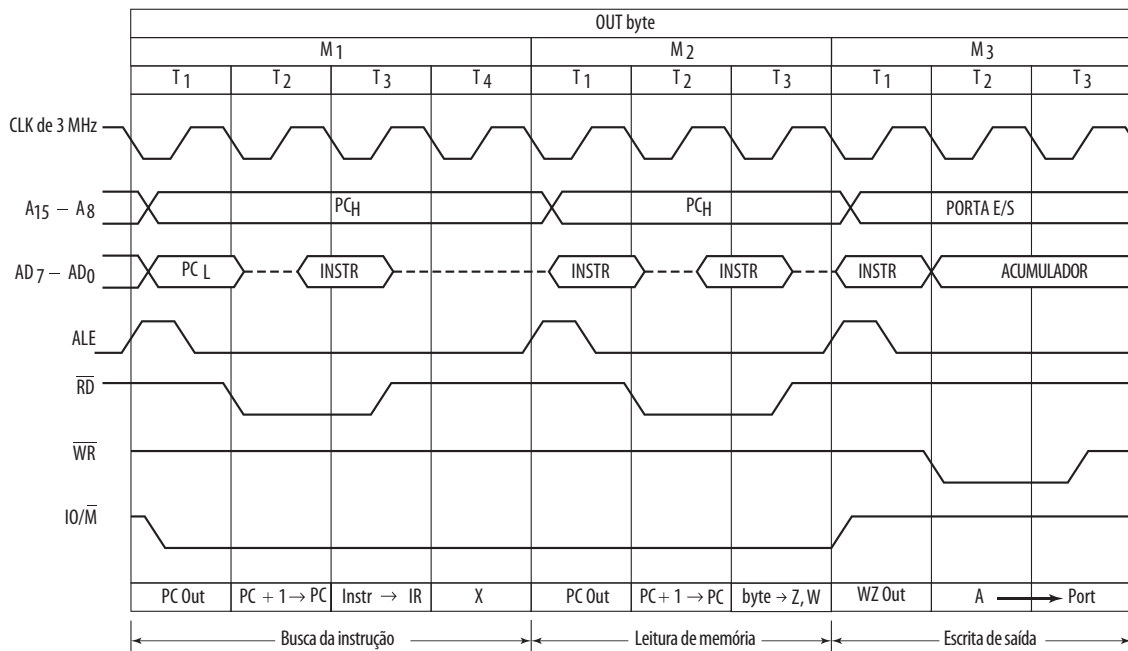
Sinais de endereços e dados	
Endereços altos (A15-A8)	8 bits superiores de um endereço de 16 bits.
Endereço/Dados (AD7-AD0)	8 bits inferiores de um endereço de 16 bits ou 8 bits de dados. Esta multiplexação economiza pinos.
Dados de entrada serial (SID)	Uma entrada de um único bit para acomodar dispositivos que transmitem serialmente (um bit por vez).
Dados de saída serial (SOD)	Uma saída de um único bit para acomodar dispositivos que recebem serialmente.
Sinais de temporização e controle	
CLK (OUT)	O clock do sistema. Sinal CLK vai para chips periféricos e sincroniza as suas temporizações.
X1, X2	Estes sinais vêm de um cristal externo ou outro dispositivo para controlar o gerador de clock interno.
Habilitar Latch de endereços (ALE)	Ocorre durante o primeiro estado do clock de um ciclo de máquina e faz com que os chips periféricos armazenem linhas de endereços. Isto permite que o módulo de endereço (por exemplo, memória, E/S) reconheça que está sendo endereçado.
Estado (S0, S1)	Sinais de controle usados para indicar se está ocorrendo uma operação de leitura ou escrita.
IO/M	Usado para habilitar módulo de E/S ou de memória para operações de leitura e escrita.
Controle de leitura (RD)	Indica que a memória selecionada ou módulo de E/S está para ser lido e que o barramento de dados está disponível para transferência de dados.
Controle de escrita (WR)	Indica que os dados no barramento de dados estão para ser escritos na posição de memória ou E/S selecionada.
Símbolos iniciados pela memória ou pela E/S	
Espera (HOLD)	Requisita a CPU que abandone o controle e o uso do barramento externo de sistema. A CPU irá completar a execução da instrução atualmente em IR e depois irá entrar em um estado de espera durante o qual nenhum sinal é inserido pela CPU para os barramentos de controle, endereços ou dados. Durante o estado de espera, o barramento pode ser usado para operações DMA.
Reconhecimento de espera (HOLDA)	Este sinal de controle da unidade de controle que reconhece o sinal HOLD e indica que o barramento está disponível agora.
READY (PRONTO)	Usado para sincronizar a CPU com dispositivos de memória ou E/S mais lentos. Quando um dispositivo endereçado envia um READY, a CPU pode proceder com uma operação de entrada (DBIN) ou saída (WR). Caso contrário, a CPU entra em um estado de espera até que o dispositivo esteja pronto.
Sinais relacionados com interrupções	
TRAP	Interrupções de reinicialização (RST 7.5, 6.5, 5.5).
Requisição de interrupção (INTR)	Estas cinco linhas são usadas por um dispositivo externo para interromper a CPU. Ela não atenderá a requisição se estiver no estado de espera ou se a interrupção estiver desabilitada. Uma interrupção é atendida apenas na conclusão de uma instrução. As interrupções estão na ordem descendente de prioridade.
Reconhecimento	Reconhece uma interrupção.
Inicialização de CPU	
RESET IN	Faz o conteúdo de PC ser definido para zero. A CPU continua a execução na posição zero.
RESET OUT	Reconhece que a CPU foi reiniciada. O sinal pode ser usado para reiniciar o restante do sistema.
Voltagem e aterramento	
VCC	Fonte de alimentação de +5 volts.
VSS	Aterramento elétrico.

Figura 15.8 Configuração de pinos de Intel 8085

A temporização das operações do processador é sincronizada pelo clock e controlada pela unidade de controle por meio de sinais de controle. Cada ciclo de instrução é dividido em um até cinco *ciclos de máquina*; cada ciclo de máquina por sua vez é dividido em três a cinco *estados*. Cada estado dura um ciclo de clock. Durante um estado, o processador executa uma ou um conjunto de micro-operações simultâneas conforme determinado pelos sinais de controle.

O número de ciclos de máquina é fixo para uma determinada instrução, mas varia de uma instrução para outra. Os ciclos de máquina são definidos para serem equivalentes a acessos ao barramento. Assim, o número de ciclos de máquina para uma instrução depende do número de vezes que o processador precisa se comunicar com dispositivos externos. Por exemplo, se uma instrução consiste de duas partes de 8 bits, então dois ciclos de máquina são necessários para obter a instrução. Se essa instrução envolve uma operação de memória ou E/S de 1 byte, então um terceiro ciclo de máquina é necessário para execução.

A Figura 15.9 mostra um exemplo de temporização do 8085, mostrando o valor dos sinais de controle externos. É claro que, ao mesmo tempo, a unidade de controle gera sinais de controle internos que controlam transferências de dados internas. O diagrama mostra o ciclo de instrução para uma instrução OUT. Três ciclos de máquina (M_1 , M_2 , M_3) são necessários. A instrução OUT é obtida durante o primeiro ciclo. O segundo ciclo de máquina obtém a segunda metade da instrução, a qual contém o número do dispositivo I/O selecionado para saída. Durante o terceiro ciclo, o conteúdo de AC é escrito no dispositivo selecionado através do barramento de dados.

Figura 15.9 Diagrama de temporização para instrução OUT do Intel 8085

O pulso Habilitar Latch de Endereço (ALE, do inglês *address latch enable*) sinaliza o início de cada ciclo de máquina a partir da unidade de controle. Ele alerta os circuitos externos. Durante o estado de tempo T_1 do ciclo de máquina M_1 , a unidade de controle define o sinal IO/M para indicar que se trata de uma operação de memória. Além disso, a unidade de controle faz com que o conteúdo de PC seja colocado no barramento de endereços (A_{15} até A_8) e no barramento de endereços/dados (AD_7 até AD_0). Na borda de descida do pulso ALE, os outros módulos no barramento armazenam o endereço.

Durante o estado de tempo T_2 , o módulo de memória endereçado coloca o conteúdo da posição de memória endereçada no barramento de endereços/dados. A unidade de controle define o sinal de Controle de Leitura (RD) para indicar uma leitura, porém aguarda até T_3 para copiar dados do barramento. Isso dá tempo ao módulo de memória para colocar dados no barramento e aos níveis de sinal para se estabilizarem. O estado final T_4 é o estado de barramento ocioso durante o qual o processador decodifica a instrução. Os ciclos de máquina restantes procedem de maneira semelhante.



15.3 Implementação por hardware

Discutimos a unidade de controle em termos de suas entradas, saídas e funções. Abordaremos agora o assunto sobre a implementação da unidade de controle. Uma grande variedade de técnicas têm sido usadas. A maioria se enquadra em uma das duas categorias:

- Implementação por hardware.
- Implementação microprogramada.

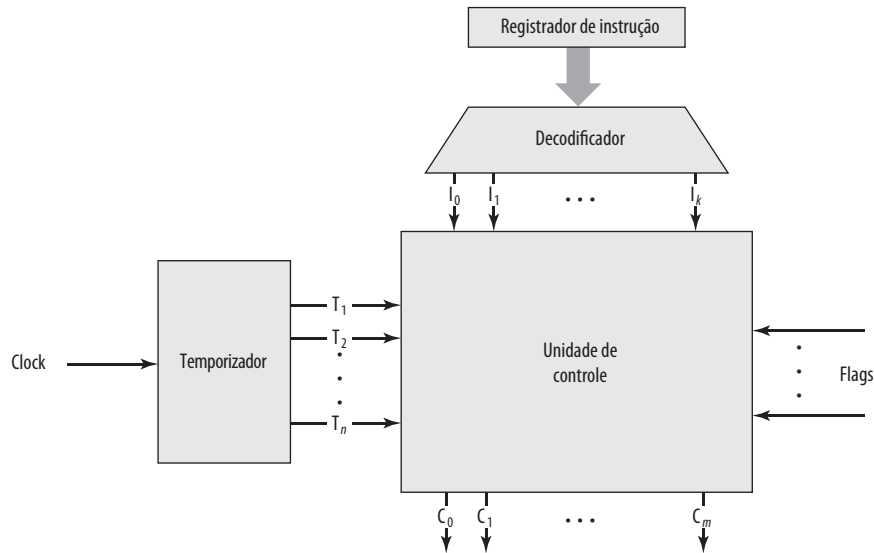
Em uma implementação por hardware, a unidade de controle é basicamente um circuito que implementa uma máquina de estado. Seus sinais lógicos de entrada são transformados em um conjunto de sinais lógicos de saída, que são os sinais de controle. Esta abordagem é analisada nesta seção. A implementação microprogramada é assunto do Capítulo 16.



Entradas da unidade de controle

A Figura 15.4 ilustra a unidade de controle da forma que a discutimos até agora. As principais entradas são o registrador de instrução, o clock, os flags e os sinais do barramento de controle. No caso das flags e dos sinais do

[illegible]

Figura 15.10 Unidade de controle com entradas decodificadas

PQ = 00	Ciclo de busca
PQ = 01	Ciclo indireto
PQ = 10	Ciclo de execução
PQ = 11	Ciclo de interrupção

Então, a seguinte expressão lógica define C_5 :

$$C_5 = \bar{P} \cdot \bar{Q} \cdot T_2 + \bar{P} \cdot Q \cdot T_2$$

Ou seja, o sinal de controle C_5 será definido durante a segunda unidade de tempo do ciclo de busca e do ciclo indireto.

Esta expressão não está completa. C_5 é necessário também durante o ciclo de execução. Para nosso simples exemplo, vamos supor que existam apenas três instruções que leem da memória: LDA, ADD e AND. Agora definimos C_5 como:

$$C_5 = \bar{P} \cdot \bar{Q} \cdot T_2 + \bar{P} \cdot Q \cdot T_2 + P \cdot \bar{Q} \cdot (LDA + ADD + AND) \cdot T_2$$

Este mesmo processo poderia ser repetido para cada sinal de controle gerado pelo processador. O resultado seria um conjunto de equações lógicas que definem o comportamento da unidade de controle e do processador.

Para juntar tudo, a unidade de controle deve controlar o estado do ciclo de instrução. Conforme mencionamos, ao fim de cada subciclo (busca, indireto, execução, interrupção), a unidade de controle emite um sinal que faz o temporizador reiniciar e gere T_1 . A unidade de controle precisa também definir os valores apropriados de P e Q para definir o próximo subciclo a ser executado.

O leitor deveria ser capaz de compreender que, em um processador moderno complexo, o número de equações lógicas necessário para definir a unidade de controle é muito grande. A tarefa de implementar um circuito combinatório que satisfaça todas essas equações se torna extremamente difícil. O resultado é que uma abordagem bem mais simples, conhecida como *microprogramação*, normalmente é usada. Este é o assunto do próximo capítulo.



15.4 Leitura recomendada

Uma série de livros trata dos princípios básicos da função da unidade de controle; dois tratamentos bastantes esclarecedores são Farhat (2004^a) e Mano (2004^b).

Principais termos, perguntas de revisão e problemas

Principais termos

Barramento de controle	Sinal de controle	Implementação por hardware
Caminho de controle	Unidade de controle	Micro-operações

Perguntas de revisão

- 15.1 Explique a diferença entre a sequência de escrita e a sequência de tempo de uma instrução.
- 15.2 Qual é a relação entre instruções e micro-operações?
- 15.3 Qual é a função geral de uma unidade de controle do processador?
- 15.4 Defina um processo em três passos que leva à caracterização da unidade de controle.
- 15.5 Quais tarefas básicas uma unidade de controle efetua?
- 15.6 Forneça uma lista típica de entradas e saídas de uma unidade de controle.
- 15.7 Relacione três tipos de sinais de controle.
- 15.8 Explique resumidamente o que significa uma implementação por hardware de uma unidade de controle.

Problemas

- 15.1 Sua ALU pode adicionar seus dois registradores de entrada e pode logicamente complementar os bits de cada um dos registradores de entrada, mas não pode subtrair. Números devem ser armazenados na representação de complemento de dois. Quais micro-operações a sua unidade de controle deve efetuar para fazer a subtração.
- 15.2 Mostre as micro-operações e os sinais de controle da mesma forma como na Tabela 15.1 para o processador da Figura 15.5 para seguintes instruções:
 - Carregar acumulador.
 - Armazenar acumulador.
 - Adicionar para acumulador.
 - AND para acumulador.
 - Salto.
 - Salto se $AC = 0$.
 - Complementar acumulador.
- 15.3 Suponha que os atrasos de propagação pelo barramento e pela ALU da Figura 15.6 sejam de 20 e 100 ns, respectivamente. O tempo necessário para um registrador copiar dados do barramento é 10 ns. Qual é o tempo que deve ser permitido para
 - a. transferir dados de um registrador para outro?
 - b. incrementar o contador de programa?
- 15.4 Escreva a sequência de micro-operações necessária para a estrutura de barramento da Figura 15.6 adicionar um número para AC quando o número for:
 - a. um operando imediato.
 - b. um operando de endereço direto.
 - c. um operando de endereço indireto.
- 15.5 Uma pilha é implementada conforme mostrado na Figura 10.14. Mostre a sequência de micro-operações para:
 - a. tirar elemento da pilha.
 - b. colocar elemento na pilha.

Referências

- a FARHAT, H. *Digital design and computer organization*. Boca Raton, FL: CRC Press, 2004.
- b MANO, M. *Logic and computer design fundamentals*. Upper Saddle River, NJ: Prentice Hall, 2004.